
Metal-Sci: A Scientific Compute Benchmark for Evolutionary LLM Kernel Search on Apple Silicon

Anonymous Authors
Anonymous Affiliation
anonymous@example.org

Abstract

LLM-driven code search has produced impressive results on GPU ML kernels (GEMM, attention, convolution), but the optimization patterns in scientific computing are systematically different: bandwidth-bound stencils with halo tiling, register-blocked all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics with atomics, parallel-chain MCMC, multi-kernel non-linear PDE solvers, and butterfly/data-shuffle spectral transforms. We present METAL-SCI, a 10-task benchmark of scientific Apple-Silicon Metal compute kernels spanning six optimization regimes. Every task ships a CPU reference, a roofline ceiling, and a naive but correct seed; the fitness signal is the geometric mean of *achieved/ceiling* across three problem sizes, with any tolerance failure forcing the score to zero. We release the harness and seeds, and report preliminary 5–25 iteration evolution runs for Claude Opus 4.7, Gemini 3-Flash, and Gemini 3.1-Pro on an Apple M1 Pro. Frontier models recover canonical optimisations on well-trodden tasks (threadgroup tiling on N-body, BGK symmetry algebra on Lattice Boltzmann, template specialisation on HMC) yet still mishandle Metal-specific concurrency and naming syntax and stall at single-digit percent of peak on irregular-memory tasks (Lennard-Jones with cell lists). On 3D FFT we observe a clean dichotomy: Opus 4.7 recovers the CUDA-style Stockham + twiddle-table recipe but sacrifices threadgroup occupancy ($1.03\times$ over the seed), while Gemini-3.1-Pro reaches for Metal-native `simd_shuffle_xor` intra-simdgroup butterflies and wins ($1.19\times$, $2\text{--}2.4\times$ ahead of Opus at the cache-resident sizes). The benchmark is designed for fast iteration: each candidate compiles, runs, verifies, and scores in seconds.

1 Introduction

LLM code-search systems — FunSearch [1], AlphaEvolve [2], and recent kernel-generation work [3] — evaluate language models as *optimisers* of executable artefacts. The choice of artefact matters: KernelBench targets PyTorch ML kernels (GEMM, attention, convolution) on CUDA, where canonical implementations are heavily represented in pretraining data and the optimisation surface is dominated by tiling and warp-level reductions.

Scientific computing exposes a different surface. Stencils are bandwidth-bound and reward halo and temporal blocking. All-pairs interactions are compute-bound and reward register blocking with shared-memory cooperative loads. Lattice Boltzmann ping-pongs nine distribution functions per cell with non-trivial pull-stream indexing. Cell-list molecular dynamics touches irregular memory through atomic counters. Hamiltonian Monte Carlo runs many chains in parallel where register pressure scales with the problem dimension d . Grad-Shafranov solvers chain a max-reduction with a variable-coefficient stencil. 3D FFTs are dominated by data-shuffle/butterfly patterns and twiddle-factor caching rather than tiling. Each pattern stresses a distinct dimension of the compiler/memory hierarchy, and canonical optimisations [5,6,7,8,11] differ regime-to-regime.

We target Apple Silicon’s Metal compute pipeline. It is underrepresented in CUDA-centric training data — frontier models reliably mishandle Metal-specific syntax such as the `[[max_total_threads_per_threadgroup]]` kernel attribute, a real out-of-distribution generalisation test. Its unified memory model removes host-device copy plumbing, enabling sub-second compile-run-verify cycles per candidate that are essential for fast evolutionary loops. And it supports rich simdgroup intrinsics (`simd_max`, `simd_broadcast`) and threadgroup-memory tiling, giving the LLM a non-trivial optimisation surface.

Contributions. (i) A 10-task scientific compute benchmark over six optimisation regimes, each with a CPU reference, roofline-anchored fitness, and multi-size generalisation gate. (ii) A runtime-compiled harness that exposes compile errors, correctness diagnostics, and per-size GPU timing back to the LLM. (iii) Preliminary evolution runs across Claude Opus 4.7 and Gemini 3 on M1 Pro, qualitatively analysing which patterns each model recovers.

2 Benchmark tasks

Each task ships (a) a Metal source describing the problem, (b) a naive correct seed, (c) a CPU reference with task-specific tolerance, (d) three in-distribution sizes plus one held-out size, and (e) a roofline ceiling in either GFLOPS (compute-bound) or GB/s (bandwidth-bound). Tasks are grouped by optimisation regime.

2.1 Regular stencils (regime R1)

heat2d. Two-dimensional heat equation, 5-point stencil on $(N_x \times N_y)$ grid with Dirichlet boundaries. Per timestep, per interior cell:

$$u_{i,j}^{n+1} = u_{i,j}^n + \alpha(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n). \quad (1)$$

Bandwidth-bound at 8 B/cell; tests halo handling and temporal blocking.

wave3d. Three-dimensional acoustic wave equation, 7-point isotropic Laplacian, leapfrog in time:

$$u_{i,j,k}^{n+1} = 2u_{i,j,k}^n - u_{i,j,k}^{n-1} + \alpha(u_{i\pm 1,j,k}^n + u_{i,j\pm 1,k}^n + u_{i,j,k\pm 1}^n - 6u_{i,j,k}^n), \quad (2)$$

where each $\pm$ expands to two terms. CFL stability requires $\alpha = (c\Delta t/\Delta x)^2 < 1/3$ (we use 0.18). 12 B/cell unique DRAM traffic. Tests register pressure, 2.5D blocking, and marching-axis choice.

2.2 Compute-bound (regime R2)

nbody. All-pairs gravitational N -body with leapfrog. Per body i per step:

$$\mathbf{a}_i = G \sum_j m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \varepsilon^2)^{3/2}}, \quad \mathbf{v} += \mathbf{a}\Delta t, \quad \mathbf{r} += \mathbf{v}\Delta t. \quad (3)$$

Self-interaction is masked by the softening ε . ~ 20 FLOPs per pair; ceiling at peak FP32 GFLOPS. Tests register blocking and threadgroup cooperative loads.

hmc. Hamiltonian Monte Carlo on an anisotropic Gaussian target, $U(q) = \frac{1}{2}q^\top Aq$ with $A = \Sigma^{-1}$. One thread per chain; many chains in parallel. Each HMC step draws $p \sim \mathcal{N}(0, I)$ and runs L leapfrog inner steps with stepsize ε ,

$$p \leftarrow p - \frac{\varepsilon}{2}Aq, \quad q \leftarrow q + \varepsilon p, \quad p \leftarrow p - \varepsilon Aq, \dots, \quad p \leftarrow p - \frac{\varepsilon}{2}Aq, \quad (4)$$

followed by a Metropolis accept/reject with $\log u_{\text{acc}} < -\Delta H$. Correctness is verified statistically (sample mean and Frobenius covariance error vs. the target). Three sizes $(d, K) \in \{(8, 16K), (16, 4K), (32, 1K)\}$ probe the register-pressure boundary; at $d=32$ per-thread state (~ 512 B) competes with the register file.

2.3 Multi-field, exotic memory (regime R3)

lbm. D2Q9 Lattice Boltzmann, fused pull-stream + BGK collision, periodic BC. With velocity table $\mathbf{c}_k$ and weights w_k , per cell per step:

$$f_k^{\text{str}}(\mathbf{x}) = f_k^{\text{in}}(\mathbf{x} - \mathbf{c}_k), \quad \rho = \sum_k f_k^{\text{str}}, \quad \mathbf{u} = \frac{1}{\rho} \sum_k \mathbf{c}_k f_k^{\text{str}}, \quad (5)$$

$$f_k^{\text{eq}} = w_k \rho \left(1 + 3(\mathbf{c}_k \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{c}_k \cdot \mathbf{u})^2 - \frac{3}{2}\|\mathbf{u}\|^2 \right), \quad (6)$$

$$f_k^{\text{out}} = f_k^{\text{str}} - \tau^{-1}(f_k^{\text{str}} - f_k^{\text{eq}}). \quad (7)$$

Storage is SoA: $f[k N_x N_y + j N_x + i]$. 72 B/cell DRAM traffic. Tests AoS/SoA layout, push vs pull streaming, and algebraic factorisations of the BGK kernel.

ising. Two-dimensional Ising model, checkerboard Metropolis Monte Carlo on a periodic $N_x \times N_y$ lattice with $\sigma \in \{\pm 1\}$ stored as int8. Per attempt at site (i, j) :

$$h = \sigma_{i\pm 1, j} + \sigma_{i, j\pm 1} \in \{-4, \dots, 4\}, \quad \Delta E = 2J\sigma h, \quad \text{accept} \iff u < \exp(-\beta \Delta E). \quad (8)$$

A precomputed five-entry p_{accept} table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement: verification is byte-equality on the spin array. 2 B/site/sweep ceiling.

2.4 Irregular memory and atomics (regime R4)

lj. Lennard-Jones molecular dynamics with a cell-list spatial hash. Three kernels per step — `clear_cells`, `build_cells` (atomic scatter), `step` (27-neighbour-cell pair force + integration):

$$\mathbf{F}_i = \sum_{j \in \mathcal{N}(i)} -24(2r_{ij}^{-12} - r_{ij}^{-6}) r_{ij}^{-2} (\mathbf{r}_j - \mathbf{r}_i), \quad r_{ij} < r_{\text{cut}} = 2.5, \quad (9)$$

with minimum-image periodic wrap. Cell occupancy is built via `atomic_fetch_add` on a per-cell counter. Tests load balancing, atomic contention, and the irregular access patterns of neighbour-list MD.

2.5 Multi-kernel reductions (regime R5)

gradshaf. Grad-Shafranov fixed-boundary equilibrium via Picard iteration, two kernels per outer step: an interior max-reduction $\psi_{\text{axis}} = \max_{(i,j) \in \text{int}} \psi$, then a variable-coefficient 5-point stencil with nonlinear source:

$$\psi_{\text{norm}} = \psi / \psi_{\text{axis}}, \quad J = R p_{\text{axis}} \cdot 4\psi_{\text{norm}}(1 - \psi_{\text{norm}}) \cdot \mathbf{1}[0 < \psi_{\text{norm}} < 1], \quad (10)$$

$$\Delta^* \psi = a_W \psi_W + a_E \psi_E + a_N \psi_N + a_S \psi_S + a_C \psi_C, \quad (11)$$

$$\psi^{n+1} = \psi^n + \omega(-\mu_0 R J - \Delta^* \psi) / a_C, \quad (12)$$

with R -dependent $a_{W,E} = 1/dR^2 \pm 1/(2RdR)$, $a_{N,S} = 1/dZ^2$, $a_C = -2(1/dR^2 + 1/dZ^2)$. Tests in-kernel reduction strategies (single-TG vs simdgroup-tree) and multi-kernel composition.

2.6 Data-shuffle / butterfly (regime R6)

fft3d. 3D complex-to-complex forward FFT, fp32, on a power-of-two cube of side N . Convention is unnormalised (matches `numpy.fft.fftn`); storage is row-major `float2[N][N][N]`. Three named kernels — `fft3d_x`, `fft3d_y`, `fft3d_z` — are dispatched in sequence with two ping-ponged buffers, each kernel performing a length- N 1D FFT per threadgroup of N cooperating threads:

$$Y[k_1, k_2, k_3] = \sum_{n_1, n_2, n_3=0}^{N-1} X[n_1, n_2, n_3] \exp\left(-\frac{2\pi i}{N}(k_1 n_1 + k_2 n_2 + k_3 n_3)\right). \quad (13)$$

Sizes $N \in \{32, 64, 128\}$ cover three working-set regimes: 32^3 (~ 256 KB) is SLC-resident and compute-bound; 128^3 (~ 16 MB) DRAM-binds. The optimisation surface is dominated by data movement, not arithmetic — bit-reversal vs Stockham auto-sort, twiddle-factor caching, mixed-radix (radix-4, radix-8) butterflies for fewer barriers, and `simd_shuffle` / `simd_shuffle_xor` for

Table 1: Preliminary evolution runs on Apple M1 Pro. Score is the geometric mean of achieved/ceiling over three in-distribution sizes; correctness is a hard gate. Best @ largest is the fraction-of-ceiling at the largest in-distribution size.

Task	Model	Iters	Seed	Best	Speedup	Best @ largest
heat2d	gemini-2.5-flash	1	0.484	0.484	1.00×	—
wave3d	claude-opus-4-7	15	0.533	0.673	1.26×	77.1%
nbody	gemini-3-flash-preview	5	0.020	0.061	2.99×	17.7%
nbody	gemini-3.1-pro-preview	5	0.020	0.045	2.19×	22.0%
hmc	claude-opus-4-7	10	0.009	0.093	10.6×	21.6% (d=8)
lbm	claude-opus-4-7	25	0.395	0.576	1.46×	121.6%
lbm	claude-opus-4-6	25	0.392	0.545	1.39×	—
ising	claude-opus-4-7	10	0.066	0.075	1.13×	11.6%
lj	claude-opus-4-7	10	0.003	0.005	1.77×	1.0%
lj	gemini-3.1-pro-preview	15	0.003	0.006	1.98×	—
gradshaf	claude-opus-4-7	10	0.022	0.041	1.89×	10.7%
fft3d	claude-opus-4-7	10	0.162	0.167	1.03×	29.8%
fft3d	gemini-3.1-pro-preview	10	0.237	0.282	1.19×	31.5%

intra-simdgroup butterflies are all on the table. Tests threadgroup-memory bank-conflict avoidance and the per-axis stride asymmetry (stride 1 along x vs strides N and N^2 along y, z). $\sim 5N \log_2 N$ FLOPs per 1D FFT; effective DRAM traffic 96 B/cell across the three axis passes (16 B read + 16 B write per pass). Verification is max-norm against `numpy.fft.fftn` on a fixed seeded Gaussian input, tolerance $10^{-3} + 10^{-3} \|Y\|_\infty$.

A bandwidth-bound smoke-test (`saxpy`) is also included to validate the harness.

3 Harness

Compile and dispatch. The harness uses PyObjC’s Metal bindings and runtime-compiles `.metal` source via `MTLDevice.newLibraryWithSource`, avoiding the offline `xcrun metal` toolchain; compile errors are returned as structured strings to the LLM. Buffers are allocated in unified memory; numpy views alias buffer contents with a single copy on read-back. All dispatches for a multi-size run share one `MTLCommandBuffer`, and timings come from `GPUEndTime - GPUStartTime` (3 warmup, 10 timed, median reported). The chip is detected from `sysctl` and looked up in a per-family table (M1 through M4) for peak FP32 GFLOPS and DRAM bandwidth; all runs here are on Apple M1 Pro (4500 GFLOPS, 200 GB/s).

Scoring. For a candidate passing correctness at every size, $S = (\prod_i f_i)^{1/n}$ with $f_i = \text{achieved}_i / \text{ceiling}_i$. Any tolerance failure forces $S=0$; this hard gate prevents trading correctness for speed, and the geometric mean across sizes discourages overfitting to one regime.

Evolution loop. A single-population ($\mu=1+\lambda=1$) loop. Iteration i shows the LLM the previous candidate, the incumbent best, a short history per iteration, and the original task spec. The LLM returns a Metal source block (multi-kernel tasks emit multiple `kernel` functions in one file). Candidates strictly beating the incumbent are promoted; prompts, raw responses, extended-thinking tokens, sources, and per-size JSON results are persisted. A single `call_llm` bridge dispatches Claude via the Agent SDK and Gemini via `google-gemai`.

4 Preliminary results

Table 1 summarises one representative run per task on Apple M1 Pro. Scores are geometric mean of achieved/ceiling across the three in-distribution sizes; the column “best @ largest” reports the fraction-of-ceiling at the largest size of the incumbent best.

The LBM 256² result above 100% of the published ceiling reflects working-set residency in Apple Silicon’s system-level cache: the 8 B/cell roofline is DRAM-only, while the streamed 9-distribution

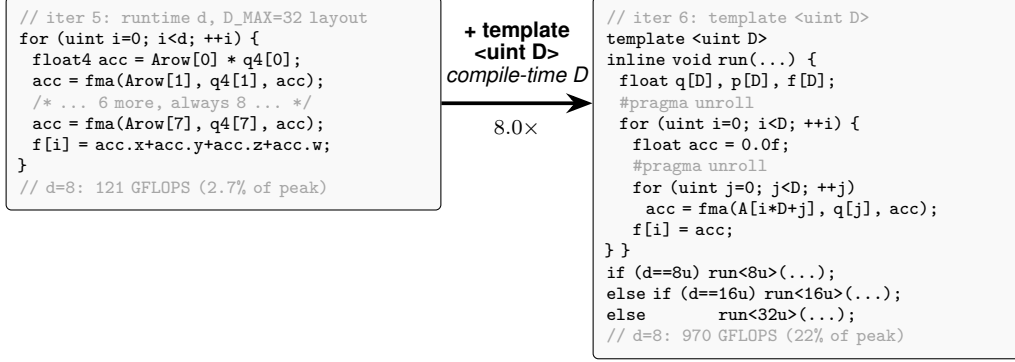


Figure 1: Paradigmatic candidate evolution: HMC, Opus 4.7, iter 5 $\rightarrow$ 6. The structural change is one declaration — `template <uint D>` with three runtime-dispatched instantiations $d \in \{8, 16, 32\}$. Iter 5 had a manually-unrolled float4 inner loop against a fixed $D_{\max}=32$ layout (over-computing at $d=8$, plus a 4-way horizontal sum); iter 6 sizes q, p, f exactly to D so both loops fully unroll into scalar FMAs — moving $d=8$ from 121 to 970 GFLOPS in one iteration after five had stalled at 2–4% of peak.

working set fits in SLC at this size. We report it unmodified to flag the limitation of the static roofline in the multi-field, multi-step regime.

Qualitative findings. Four patterns recur across runs. (i) *Models recover canonical optimisations on well-trodden tasks.* On nbody, Gemini-3.1-Pro one-shots threadgroup-memory tiling with cooperative load and 4-way ILP unroll (Nyland 2007); on LBM, both Claude 4.6 and 4.7 independently find the BGK pairwise-symmetry algebraic factorisation; on HMC (Fig. 1), Opus 4.7 introduces a `template <uint D>` worker dispatched on runtime d at iteration 6, taking $d=8$ from 121 to 970 GFLOPS in a single iteration by enabling full unroll of the inner Aq matvec. (ii) *Metal-specific syntax is a real OOD failure, and replicates across models.* On LBM, Opus 4.6 mishandles the `[[max_total_threads_per_threadgroup(N)]]` attribute placement on six independent iterations (the same compile error each time), missing a structural lever Opus 4.7 uses correctly on its first attempt. On gradshaf, an early run hit 6/10 NaN failures from a hard-coded 32×8 tile against a host-pinned 16×16 threadgroup. On fft3d, both Opus 4.7 and Gemini-3.1-Pro placed the same attribute as a *suffix* after the parameter list (between `)` and `{`) on iteration 1, an independent reproduction of the same OOD failure across model families. Two further fft3d failures were single-model: Opus 4.7 hit `uint half = N >> 1u`; on iters 1 and 6 — `half` is reserved as MSL’s fp16 type, producing a cascading parse error not seen in CUDA — and Gemini-3.1-Pro called `sincospi`, which exists in CUDA but not in MSL (only `sincos`). All four are Metal-grammar / Metal-stdlib mismatches with no analogue in CUDA training data. (iii) *Models pick different optimisation axes; Metal-native beats CUDA-style.* On fft3d, the two models diverge meaningfully. Opus 4.7 recovers the CUDA-FFT canonical recipe by iteration 10 — Stockham auto-sort, mixed radix-4 + radix-2 final stage, cooperative twiddle-table precompute — but pays for it with ~ 20 KB of threadgroup memory (two ~ 8 KB ping-pong buffers plus the twiddle table), suppressing resident-TG count and capping the gain at $1.03\times$. Gemini-3.1-Pro finds the Metal-native lever instead: `simd_shuffle_xor`-based intra-simdgroup butterflies that make the first five stages (i.e. the entire FFT for $N \leq 32$, all but 1–2 stages for $N=64, 128$) *barrier-free*, plus `reverse_bits` and `ctz` intrinsics and a $i+i/32$ bank-conflict padding scheme. Twiddles are recomputed on the fly, freeing the 4 KB Opus spent on its table. Result: $1.19\times$ over the seed and a $2.0\text{--}2.4\times$ per-size lead over Opus at 32^3 and 64^3 (21% / 34% of peak BW vs 9% / 18%), narrowing to a tie at 128^3 where DRAM dominates. The signal is sharper than “one is better”: the models are reaching for different priors — CUDA-FFT vs Metal-shuffle — and on this hardware the latter wins. (iv) *Irregular tasks remain hard.* Lennard-Jones with cell lists still sits at $\sim 1\%$ of FP32 peak after 10 iterations across both Claude and Gemini; the atomic scatter and variable-occupancy neighbour-cell loop are not patterns either model recombines effectively in our budget.

5 Discussion

A roofline-anchored score answers “how close are we to the hardware?” in physical units; the geometric mean across sizes is hard to game by overfitting one regime. Apple Silicon’s unified memory halves the host-side scaffolding compared to CUDA, enabling sub-second compile-run-verify cycles per candidate — which matters more than it sounds: an evolutionary loop with tens of candidates needs a fast *harness*, not a fast kernel. Metal’s smaller, quirrier surface (simdgroup intrinsics, threadgroup attributes) also surfaces OOD failures of CUDA-trained LLMs on tasks that are otherwise textbook.

Limitations and future work. The static per-chip ceilings do not account for SLC residency at small sizes (see the LBM 256² result above); a workload-aware roofline [4] per task and per size would tighten the score. The single-population (1+1) loop plateaus within 10–25 iterations on most tasks — an island-model or FunSearch-style [1] archive with a novelty signal [2] is the natural next step, particularly for the irregular-memory tasks. Future tasks include sparse linear algebra (SpMV, CG) and cross-chip generalisation (evolve on M2 Pro, evaluate on M3 Max).

References

- [1] Romera-Paredes, B. et al. (2024) Mathematical discoveries from program search with large language models. *Nature* **625**, 468–475.
- [2] Novikov, A. et al. (2025) AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*.
- [3] Ouyang, A. et al. (2025) KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv:2502.10517*.
- [4] Williams, S., Waterman, A. & Patterson, D. (2009) Roofline: An insightful visual performance model for multi-core architectures. *CACM* **52**(4), 65–76.
- [5] Datta, K. et al. (2008) Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. *Proc. SC '08*.
- [6] Nyland, L., Harris, M. & Prins, J. (2007) Fast N-body simulation with CUDA. *GPU Gems 3*, ch. 31.
- [7] Schönherr, M. et al. (2011) Multi-thread implementations of the lattice Boltzmann method on non-uniform grids for CPUs and GPUs. *Comput. Math. Appl.* **61**(12), 3730–3743.
- [8] Anderson, J. A., Lorenz, C. D. & Travesset, A. (2008) General purpose molecular dynamics simulations fully implemented on graphics processing units. *J. Comput. Phys.* **227**(10), 5342–5359.
- [9] Micikevicius, P. (2009) 3D finite difference computation on GPUs using CUDA. *Proc. GPGPU-2*.
- [10] Geb, L. et al. (2022) Seamless GPU acceleration for C++ based physics on the Apple M1’s unified processing units. *arXiv:2206.01791*.
- [11] Govindaraju, N. K. et al. (2008) High performance discrete Fourier transforms on graphics processors. *Proc. SC '08*.
- [12] Li, J. et al. (2025) TritonBench: Benchmarking large language model capabilities for generating Triton operators. *Findings of ACL 2025*, pp. 23053–23066.
- [13] Saroufim, M. et al. (2025) BackendBench: benchmarking PyTorch-backend kernel generation.
- [14] Wen, J. et al. (2025) MultiKernelBench: multi-platform LLM kernel generation across CUDA, Ascend C, and Pallas.
- [15] Kalade, S. & Schelle, G. (2025) NPUEval: optimising NPU kernels with LLMs and open-source compilers. *arXiv:2507.14403*.
- [16] Nie, J. et al. (2026) KernelCraft: benchmarking agentic close-to-metal kernel generation on emerging hardware. *arXiv:2603.08721*.
- [17] Lange, R. T. et al. (2025) Towards robust agentic CUDA kernel benchmarking, verification, and optimisation. *arXiv:2509.14279*.
- [18] Guo, P. et al. (2025) EvoEngineer: mastering automated CUDA kernel evolution with large language models. *arXiv:2510.03760*.

A Related work

A.1 LLM-driven kernel-generation benchmarks

A wave of benchmarks has emerged for evaluating LLMs as generators of high-performance GPU kernels. *KernelBench* [3] targets PyTorch ML kernels (GEMM, attention, convolution, normalisation, activation) on CUDA and scores single-shot generation by speedup against the PyTorch eager baseline. *TritonBench* [12] extends to the Triton DSL and adds reference code-similarity to correctness and performance. *BackendBench* [13] evaluates correctness alone for kernels written against the PyTorch backend interface. *MultiKernelBench* [14] broadens the platform set to CUDA, Ascend C, and Pallas while retaining the ML-operator workload set. *NPUEval* [15] targets the AMD AIE NPU in C++ and exposes a tool-use feedback loop with multi-turn regeneration. Most recently, *KernelCraft* [16] benchmarks bare-metal assembly kernel generation on emerging accelerator ISAs (PLENA, AMD NPU, Coral NPU, Sonic BOOM), with native tool interfaces

Table 2: METAL-SCI versus existing LLM kernel-generation benchmarks across the axes that drive the design of our harness. *Domain*: ML = neural-network operators (GEMM, attention, convolution, normalisation, activation); HPC = scientific compute (stencil, N -body, LBM, MD, MCMC, PDE solver, FFT). *Score basis*: what *achieved* is normalised against. *Multi-size*: whether the score requires generalisation across problem sizes. *Search*: the loop structure exposed to the LLM. Within the ML row, all listed benchmarks span a single optimisation regime (matmul-style, with norm/activation epilogues); METAL-SCI spans six structurally distinct regimes (R1–R6 of Section 2).

Benchmark	Target	Domain	Score basis	Multi-size	Search
KernelBench [3]	CUDA	ML	PyTorch speedup	—	single-shot
TritonBench [12]	Triton DSL	ML	speedup + code-sim	—	single-shot
BackendBench [13]	PyTorch backend	ML	correctness only	—	single-shot
MultiKernelBench [14]	CUDA / Ascend C / Pallas	ML	compiler speedup	—	multi-turn
NPUEval [15]	AIE C++ (AMD NPU)	ML	compiler speedup	—	tool-use
KernelCraft [16]	accelerator asm	ML	compiler speedup	cfg. vars	agentic tool-use
METAL-SCI (ours)	Apple Metal MSL	HPC	% of roofline	3 in + 1 held-out	evolutionary (1+1)

(`check_syntax`, `run_evaluation`, `view_output`, `grep_docs`) and a three-tier ML-task taxonomy (primitive, composite, end-to-end). Across these benchmarks the workload is ML and the headline number is speedup against a vendor or compiler baseline.

A.2 LLM-driven evolutionary code search

The broader paradigm of LLMs as *optimisers* inside evolutionary code-search loops was established by *FunSearch* [1] for symbolic-algorithm discovery and scaled by *AlphaEvolve* [2] across mathematical and algorithmic domains. *AI CUDA Engineer* [17] and *EvoEngineer* [18] specialise this paradigm to CUDA kernel optimisation, replacing single-shot or agentic regeneration with a population/archive driven by a fitness signal — closer in spirit to our ($\mu=1+\lambda=1$) loop, but on CUDA ML kernels rather than Apple Silicon scientific kernels.

A.3 Where METAL-SCI differs

Existing kernel-generation benchmarks share three structural choices that METAL-SCI deviates from. (i) *Workload domain*. All listed benchmarks target ML operators, where the optimisation surface is dominated by tiling and warp-level reduction patterns heavily represented in pretraining data. METAL-SCI targets scientific compute — regular and irregular stencils, all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics, parallel-chain MCMC, multi-kernel nonlinear PDE solvers, and butterfly spectral transforms — six structurally distinct optimisation regimes (R1–R6 of Section 2) whose canonical recipes [5,6,7,8,11] do not transfer between regimes. (ii) *Score basis*. All listed benchmarks normalise *achieved* against a vendor or compiler baseline (PyTorch eager, compiler-emitted code, expert hand-tuned reference). This couples the headline number to the strength of that baseline. METAL-SCI normalises against the per-chip roofline ceiling (peak FP32 GFLOPS, peak DRAM GB/s), a hardware-anchored target independent of any reference implementation, with a hard correctness gate ($S=0$ on tolerance failure) and a geometric mean across three problem sizes that discourages overfit to a single working-set regime. (iii) *Hardware*. No listed benchmark targets Apple Silicon’s Metal compute pipeline. Metal is structurally underrepresented in CUDA-centric pretraining data and surfaces a real out-of-distribution generalisation test on Metal-grammar and Metal-stdlib idioms — the `[[max_total_threads_per_threadgroup]]` attribute placement (Sec. 4, four independent reproductions across Opus 4.7, Opus 4.6, and Gemini-3.1-Pro), `half` as a reserved fp16 keyword, the availability of `simd_shuffle_xor` for intra-simdgroup butterflies, and the absence of `sincospi`. Table 2 summarises these axes against the comparator set.